# Reproducibility

# Reproducibility I

This section explains how to regenerate every artifact in the study from a clean checkout. The exemplar's reproducibility guarantee is structural: each result is produced by a tested function and a thin script, then injected into the manuscript by generated-variable substitution — never transcribed by hand.

## How to regenerate everything I

From the repository root:

```
# 1. Run the analysis (compiles methods, exports
artifacts, writes figure + reports)
uv run python projects/templates/template_methods_pap
er/scripts/methods_analysis.py

# 2. Run the test suite with the coverage gate
uv run pytest
projects/templates/template_methods_paper/tests \
    --cov=projects/templates/template_methods_paper/s
    rc --cov-fail-under=90

# 3. Generate and inject manuscript variables
uv run python projects/templates/template_methods_pap
er/scripts/z_generate_manuscript_variables.py

# 4. Render the manuscript
```

## How to regenerate everything II

```
uv run python scripts/pipeline/stage_03_render.py
--project templates/template_methods_paper
```

## How to regenerate everything I

Or, end to end via the orchestrated pipeline:

```
uv run python scripts/runner/execute_pipeline.py
--project templates/template_methods_paper --mode
core
```

## Generated artifact registry I

The analysis script writes the following artifacts under `projects/templates/template_methods_paper/output/`:

# Generated artifact registry I

| Artifact | Produced by |
|---|---|
| data/pbspreparation_worklist.md, data/pbspreparation_plan.csv, data/pbspreparation_graph.mmd, data/pbspreparation_plan.json | compile_method() + exporters, for PBSPreparation |
| data/sensorcalibrationsweep_worklist.md, data/sensorcalibrationsweep_plan.csv, data/sensorcalibrationsweep_graph.mmd, data/sensorcalibrationsweep_plan.json | compile_method() + exporters, for SensorCalibrationSweep |
| data/compiled_plans.json | Per-method plan summary, consumed by src/template_methods_paper/manuscript_variables.py |
| reports/gate_report.json | run_all_gates() tally across both methods |
| reports/trust_chain_report.json | append_record()/verify_chain() demonstration chain |
| figures/step_counts.png | Step-count bar chart |
| data/manuscript_variables.json | Every generated-variable value, written by z_generate_manuscript_variables.py |

## Generated artifact registry I

The `output/` tree is disposable and regenerated on every run; it is not the source of truth.

# Determinism I

- `compile_method()` is deterministic by construction: the plan hash is computed from a canonical, sort-keys JSON payload over `method_name`, `method_version`, `target`, and each scheduled step's identifying fields — never from a wall-clock timestamp or a UUID.
- `topological_order()` breaks scheduling ties by ascending `step_id`, so the same `Method` object always yields the same step order across processes and platforms.
- Yes — `src/template_methods_paper/manuscript_variables.py::generate_variables` recompiles every example method twice at manuscript-build time and compares hashes live, so this guarantee is checked on every build, not merely asserted once in a test.

## Verification (no hand-transcribed numbers) I

Every quantitative claim in sec. 4 is either a generated variable sourced from a live analysis output or registered in `data/claim_ledger.yaml` for evidence-registry validation. The manuscript intentionally does not hand-transcribe volatile values, so prose and artifacts cannot disagree. Configuration provenance is itself injected: `c4e86321fa0e9cb3` is the SHA-256 of `manuscript/config.yaml` at build time, and `2026-10-29T15:39:02Z` records when the variables were generated (honoring `SOURCE_DATE_EPOCH` for byte-reproducible builds).